Course Name:	**Competitive Programming Lab**	**Semester:**	**IV**
Date of Performance:	**02 / 04 / 2026**	**DIV/ Batch No:**	**A-3**
Student Name:	**Shrived Deshmukh**	**Roll No:**	**71**

Experiment No: 6

Title: Graph Algorithms

Aim of the Experiment: To understand and use graph algorithms to solve the given problem

Objective of the Experiment:
To model and solve application-driven computational problems using graphs.

COs to be achieved:

CO 2 Model and solve computational problems using graphs, by selecting appropriate graph representations and applying graph-based reasoning to real applications.

Theory:

In computational problem-solving, a graph is a mathematical structure used to model pairwise relations between objects. A graph G = (V, E) consists of a set of vertices (or nodes) V and a set of edges E that connect these vertices. Graphs are essential for modeling real-world networks, such as city maps, social media connections, and computer networks.

Some applications of graphs in various domains:

Sr. no	Domain / Industry	Key Algorithms Used	Real-World Application
1	**Navigation & Mapping (Google Maps, Uber)**	Dijkstra's Algorithm, A* Search, Bellman-Ford	Calculating the fastest or shortest delivery route, estimating ETA.
2	**Social Networks (LinkedIn, Instagram)**	Breadth-First Search (BFS), Strongly Connected Components (Tarjan's/Kosaraju's)	"People you may know" recommendations, finding degrees of separation, identifying communities.
3	**OS & Task Scheduling (Linux, npm,**	Topological Sorting, Depth-First Search (DFS), Kahn's Algorithm	Resolving software package dependencies, CPU process scheduling, course prerequisites.

	University)		
4	**Computer Networks** **(Internet, Telecom)**	Minimum Spanning Tree (Kruskal's / Prim's), Dijkstra's (OSPF protocol)	Routing data packets efficiently, designing LAN cable layouts with minimal wire length.
5	**E-Commerce & Streaming** **(Amazon, Netflix)**	Bipartite Matching, Graph Traversal (DFS / BFS)	Recommending products or movies based on user behavior ("Users who bought X also bought Y").

-

Problem Statements:

Difficulty (as per lab)	Problem	LeetCode mapping
Easy	Path checking problem	LC 1971 – Check if path exists
Medium	Topological sorting	LC 210 – Course Schedule II
Difficult	Cycle detection (directed)	LC 207 – Course Schedule

Code :

Easy- Path checking problem
Code:-

Code

C++ Auto

```cpp
class Solution {
public:
    bool validPath(int n, vector<vector<int>>& edges, int source, int destination) {
        // Build adjacency list
        vector<vector<int>> adj(n);
        for (auto &e : edges) {
            adj[e[0]].push_back(e[1]);
            adj[e[1]].push_back(e[0]);
        }

        // BFS
        vector<bool> visited(n, false);
        queue<int> q;
        q.push(source);
        visited[source] = true;

        while (!q.empty()) {
            int node = q.front(); q.pop();
            if (node == destination) return true;
            for (int neighbor : adj[node]) {
                if (!visited[neighbor]) {
                    visited[neighbor] = true;
                    q.push(neighbor);
                }
            }
        }

        return false;
    }
};
```

Testcase | Test Result

Case 1 Case 2 +

n =

3

edges =

[[0,1],[1,2],[2,0]]

source =

0

destination =

2

Testcase | Test Result

Case 1 Case 2 +

n =

6

edges =

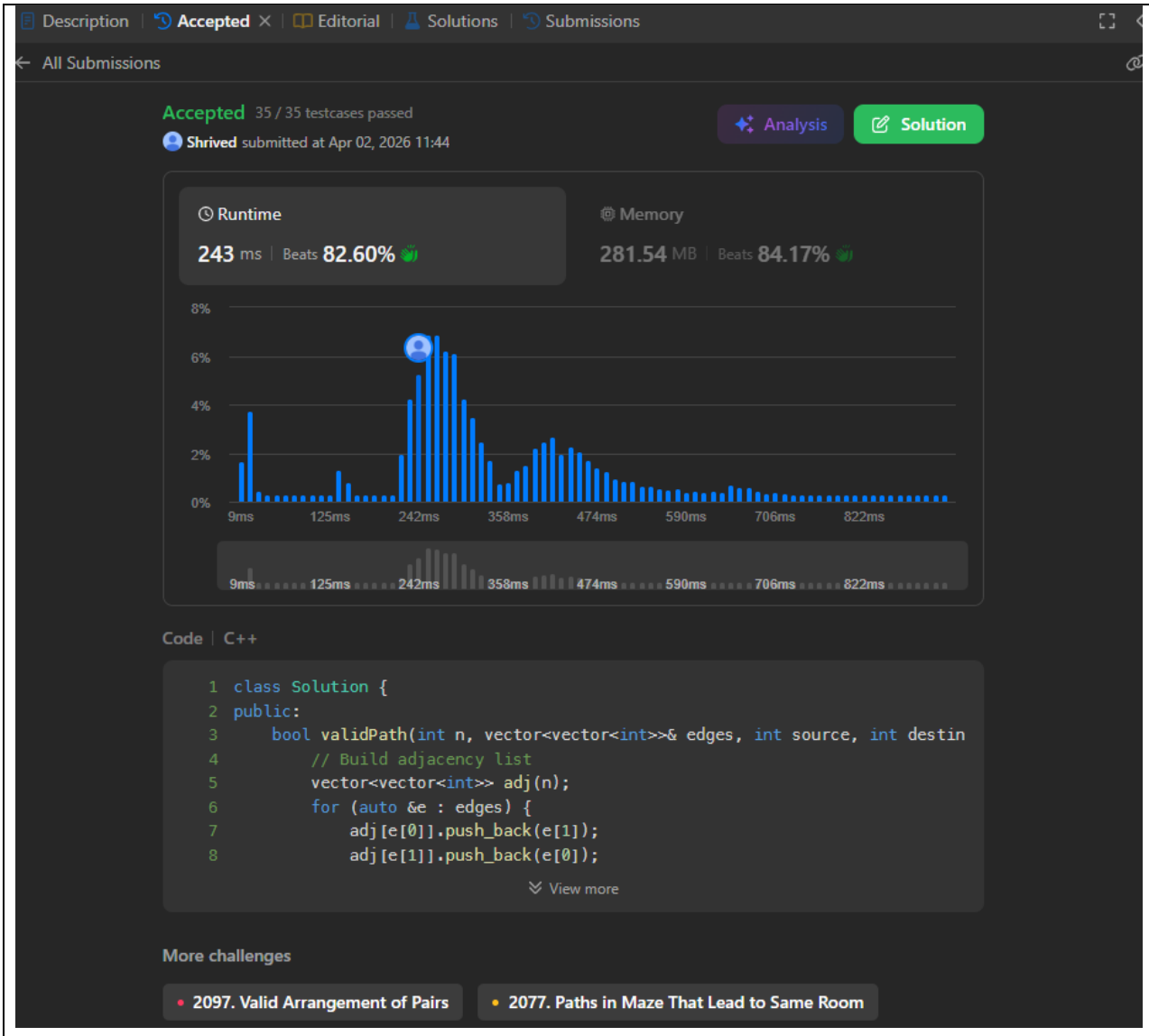
[[0,1],[0,2],[3,5],[5,4],[4,3]]

source =

0

destination =

5



Q2)Medium:-Topological Sorting:-

Code

C++ Auto

```cpp
class Solution {
public:
    vector<int> findOrder(int numCourses, vector<vector<int>>& prerequisites) {
        vector<vector<int>> adj(numCourses);
        vector<int> indegree(numCourses, 0);

        // Build adjacency list and compute indegrees
        for (auto &pre : prerequisites) {
            int u = pre[1], v = pre[0];
            adj[u].push_back(v);
            indegree[v]++;
        }

        queue<int> q;
        // Add courses with no prerequisites
        for (int i = 0; i < numCourses; i++) {
            if (indegree[i] == 0) q.push(i);
        }

        vector<int> order;

        while (!q.empty()) {
            int course = q.front(); q.pop();
            order.push_back(course);

            for (int neighbor : adj[course]) {
                indegree[neighbor]--;
                if (indegree[neighbor] == 0) {
                    q.push(neighbor);
                }
            }
        }

        // If we included all courses, return order; otherwise return empty
        if (order.size() == numCourses) return order;
        return {};
    }
};
```

Testcase | Test Result

Case 1 Case 2 Case 3 +

numCourses =

2

prerequisites =

[[1,0]]

Testcase | Test Result

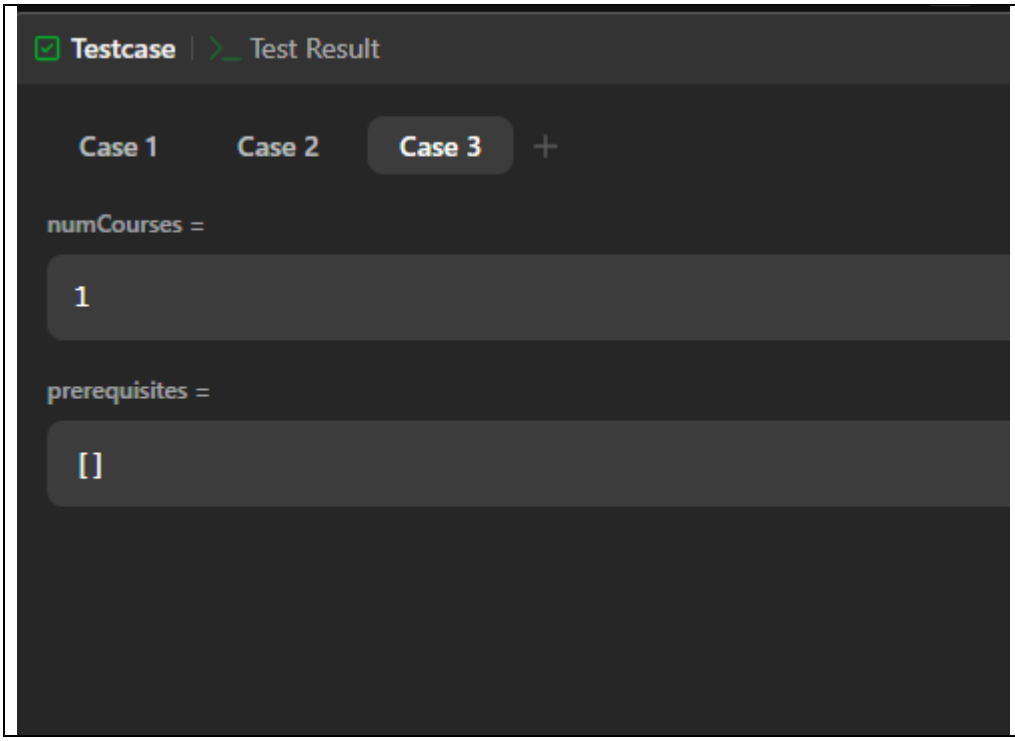
Case 1 Case 2 Case 3 +

numCourses =

4

prerequisites =

[[1,0],[2,0],[3,1],[3,2]]


Testcase
Test Result
Case 1
Case 2
Case 3
numCourses =
1
prerequisites =
[]

Description | Accepted | Editorial | Solutions | Submissions

All Submissions

Accepted 47 / 47 testcases passed

Shrived submitted at Apr 02, 2026 11:48

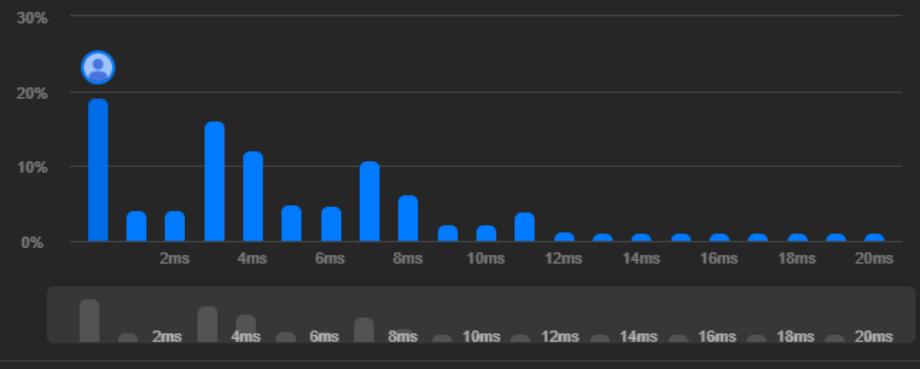
Analysis Solution

Runtime

0 ms | Beats 100.00%

Memory

18.25 MB | Beats 71.05%



Code | C++

```
class Solution {
public:
    vector<int> findOrder(int numCourses, vector<vector<int>>& prerequisites
        vector<vector<int>> adj(numCourses);
        vector<int> indegree(numCourses, 0);

        // Build adjacency list and compute indegrees
        for (auto &pre : prerequisites) {
```

View more

More challenges

- 444. Sequence Reconstruction
- 1136. Parallel Courses
- 2392. Build a Matrix With Conditions

Q3)Hard:- Cycle detection (directed)

Code

C++ Auto

```cpp
class Solution {
public:
    bool dfs(int course, vector<vector<int>> &adj, vector<int> &state) {
        if (state[course] == 1) return true;  // cycle detected
        if (state[course] == 2) return false; // already safe

        state[course] = 1; // mark as visiting
        for (int next : adj[course]) {
            if (dfs(next, adj, state)) return true;
        }
        state[course] = 2; // mark as safe
        return false;
    }

    bool canFinish(int numCourses, vector<vector<int>>& prerequisites) {
        vector<vector<int>> adj(numCourses);
        for (auto &pre : prerequisites)
            adj[pre[1]].push_back(pre[0]);

        vector<int> state(numCourses, 0); // 0=unvisited,1=visiting,2=safe
        for (int i = 0; i < numCourses; i++) {
            if (state[i] == 0 && dfs(i, adj, state))
                return false; // cycle exists
        }
        return true; // no cycle, all courses can be finished
    }
};
```

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

numCourses =

2

prerequisites =

[[1,0]]

Output

true

Expected

true

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

numCourses =

2

prerequisites =

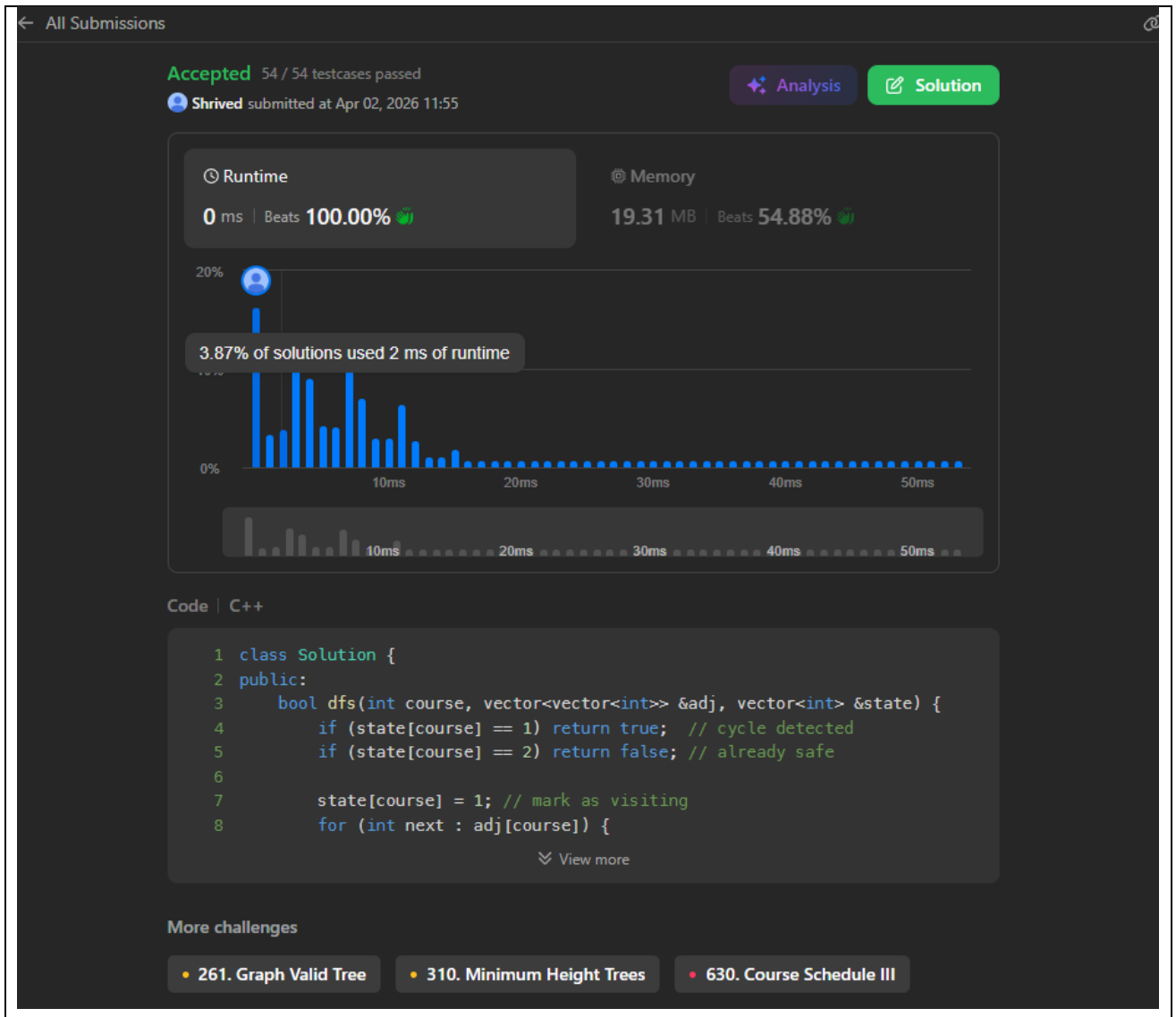
[[1,0],[0,1]]

Output

false

Expected

false



Post Lab Subjective/Objective type Questions:

1. Both Depth-First Search (DFS) and Breadth-First Search (BFS) can successfully count the connected components. However, if a problem asks you to find the *shortest path* from an entry point to an exit point in an unweighted 2D maze, which traversal algorithm must you use and why?

   Both Depth-First Search (DFS) and Breadth-First Search (BFS) can be used to explore a graph and count connected components, but when it comes to finding the shortest path in an unweighted 2D maze from an entry point to an exit point, **BFS must be used**. This is because BFS explores the graph level by level, visiting all nodes at a given distance from the start before moving to the next

level. As a result, the first time BFS reaches the exit, it guarantees that the path taken is the **shortest possible**, since any longer paths are discovered later. In contrast, DFS goes as deep as possible along one branch before backtracking, which may lead to longer or suboptimal paths being found first, and it does not guarantee the shortest path in an unweighted graph. Therefore, BFS is the correct choice for shortest-path problems in unweighted grids or mazes.

2. What is the maximum number of vertices (approximate) where an Adjacency Matrix is safe to use in standard CP platforms?

In competitive programming, an **Adjacency Matrix** uses $O(V^2)$ memory, where V is the number of vertices. Standard CP platforms usually allow **256–512 MB of memory**, and since each `int` takes 4 bytes, a matrix of size V × V uses roughly $4 \times V^2$ bytes.

- If we take 10^8 bytes as a safe upper limit (~100 MB), then:

  4×V2≤108 ⟹ V2≤2.5×107 ⟹ V≈50004 \times V^2 \le 10^8 \implies V^2 \le 2.5 \times 10^7 \implies V \approx 50004×V2≤108⟹V2≤2.5×107⟹V≈5000

So, an **Adjacency Matrix is safe for V ≈ 4,000–5,000 vertices**. Beyond this, it's better to use an **Adjacency List**, which uses O(V + E) memory and scales much better for sparse graphs.

✅**Rule of thumb:** Adjacency Matrix → only use when V ≤ 5,000; otherwise, prefer Adjacency List.

Conclusion:

An adjacency matrix is simple and allows O(1) edge lookups, but it consumes $O(V^2)$ memory. In competitive programming, it is safe to use only for graphs with around 4,000–5,000 vertices due to memory limits. For larger or sparse graphs, an adjacency list is more memory-efficient and scalable, making it the preferred choice for most practical problems.

Signature of faculty in-charge with Date: